DATA STRUCTURES – COMPLETE NOTES

Data Structures concepts for beginners, interviews, coding assessments, and technical preparation

1. What is a Data Structure?

A data structure is a way of organizing and storing data so that it can be accessed, processed, and modified efficiently. The choice of data structure affects the time and memory required by a program.

- Data structures help organize large amounts of data.
- They make operations such as searching, insertion, deletion, and sorting more efficient.
- Different problems require different data structures.
- Examples include arrays, linked lists, stacks, queues, trees, graphs, hash tables, and heaps.

2. Classification of Data Structures

Category	Types	Examples
Linear	Elements arranged sequentially	Array, Linked List, Stack, Queue
Non-linear	Elements form hierarchical/network relationships	Tree, Graph
Static	Size is generally fixed	Array
Dynamic	Size can change during execution	Linked List
Homogeneous	Elements are generally of the same type	Integer Array
Non-homogeneous	Can combine different data types	Structures/Objects

3. Algorithm and Time Complexity

An algorithm is a finite sequence of steps used to solve a problem. Time complexity describes how the running time grows as the input size increases. Space complexity describes how the additional memory requirement grows.

- O(1) – Constant time: operation does not depend on input size.
- O(log n) – Logarithmic time: input is reduced significantly at each step, as in binary search.
- O(n) – Linear time: work grows proportionally with input size.
- O(n log n) – Common in efficient comparison sorting algorithms such as merge sort.
- O(n^2) – Quadratic time: common in simple nested-loop algorithms such as bubble sort.
- O(2■) – Exponential time: grows very quickly and occurs in some recursive problems.

4. Array

An array is a linear data structure that stores elements in contiguous memory locations. Elements are accessed using an index, usually starting from 0.

```
int[] numbers = {10, 20, 30, 40};
System.out.println(numbers[2]);   // 30
```

- Fast random access using an index.
- Insertion and deletion in the middle can require shifting elements.
- In a traditional fixed-size array, the size is decided when the array is created.

- Common operations: traversal, searching, insertion, deletion, and updating.

Typical complexity:

Operation	Typical Time
Access by index	O(1)
Search (unsorted)	O(n)
Insertion at end	O(1) when space is available
Insertion/deletion in middle	O(n)

5. Linked List

A linked list consists of nodes. Each node stores data and a reference to another node. Unlike an array, linked-list nodes do not have to occupy contiguous memory.

```
class Node {
    int data;
    Node next;

    Node(int data) {
        this.data = data;
        this.next = null;
    }
}
```

Types of linked lists:

- Singly Linked List – each node points to the next node.
- Doubly Linked List – each node points to both the previous and next nodes.
- Circular Linked List – the last node points back to the first node.

Advantages:

- Dynamic size.
- Efficient insertion/deletion when the required node position is already known.
- Does not require contiguous memory.

Disadvantages:

- No direct random access by index.
- Extra memory is required for links/references.
- Traversal is generally sequential.

6. Stack

A stack is a linear data structure that follows LIFO (Last In, First Out). The last element inserted is the first element removed.

Main operations:

- push – inserts an element.
- pop – removes the top element.
- peek/top – views the top element without removing it.
- isEmpty – checks whether the stack contains no elements.

Applications:

- Function call management and recursion.
- Undo and redo operations.
- Expression evaluation.
- Parentheses matching.
- Depth First Search (DFS).

7. Queue

A queue is a linear data structure that follows FIFO (First In, First Out). The first element inserted is the first element removed.

Main operations:

- enqueue – inserts an element at the rear.
- dequeue – removes an element from the front.
- front/peek – views the front element.
- isEmpty – checks whether the queue is empty.

Types of queues:

- Simple Queue
- Circular Queue
- Priority Queue
- Deque (Double-Ended Queue)

Applications:

- CPU and task scheduling.
- Printer job management.
- Breadth First Search (BFS).
- Request handling in servers.

8. Circular Queue

A circular queue connects the rear of the queue back to the front. It can reuse positions that become empty after dequeue operations and avoids unnecessary shifting.

A circular queue is commonly implemented using front and rear indices with modulo arithmetic.

9. Deque

A deque, or double-ended queue, allows insertion and deletion from both the front and the rear.

- Insert at front
- Insert at rear
- Delete from front
- Delete from rear

10. Tree

A tree is a non-linear hierarchical data structure consisting of nodes connected by edges. The top node is called the root. A node may have child nodes.

- Root – topmost node.
- Parent – a node that has one or more children.
- Child – a node connected below a parent.
- Leaf – a node with no children.
- Height – number of edges on the longest path from a node to a leaf.
- Depth – number of edges from the root to a node.

11. Binary Tree

A binary tree is a tree in which each node has at most two children, commonly called the left child and right child.

Common forms:

- Full Binary Tree – every node has either 0 or 2 children.
- Complete Binary Tree – all levels are filled except possibly the last, and the last level is filled from left to right.
- Perfect Binary Tree – all internal nodes have two children and all leaves are at the same level.
- Balanced Binary Tree – height is kept relatively small compared with the number of nodes.

12. Binary Search Tree (BST)

A Binary Search Tree is a binary tree in which values in the left subtree are smaller than the node value and values in the right subtree are larger, subject to the chosen duplicate-handling rule.

Important property:

Inorder traversal of a valid BST produces values in sorted order.

13. Tree Traversals

Traversal	Order	Common Use
Preorder	Root → Left → Right	Copying/serialization style tasks
Inorder	Left → Root → Right	Sorted order for BST
Postorder	Left → Right → Root	Deleting/processing children first
Level Order	Level by level	Breadth-first processing

14. Heap

A heap is a specialized complete binary tree that satisfies the heap property.

- Min Heap – the smallest element is at the root.
- Max Heap – the largest element is at the root.
- Heaps are commonly used to implement priority queues.
- Heap insertion and deletion of the root are typically O(log n).
- Accessing the root is typically O(1).

15. Graph

A graph is a non-linear data structure consisting of vertices (nodes) and edges (connections). Graphs are useful for representing relationships and networks.

- Undirected Graph – edges have no direction.
- Directed Graph – edges have a direction.
- Weighted Graph – edges have associated weights/costs.
- Unweighted Graph – edges do not carry numerical weights.
- Connected Graph – paths exist between relevant pairs of vertices.
- Cyclic Graph – contains at least one cycle.

16. Graph Representation

- Adjacency Matrix – uses a 2D matrix; useful when the graph is relatively dense.
- Adjacency List – stores neighbors for each vertex; often efficient for sparse graphs.

17. Graph Traversal

Breadth First Search (BFS)

BFS explores a graph level by level. It commonly uses a queue and can find shortest paths in an unweighted graph.

Depth First Search (DFS)

DFS explores as deeply as possible before backtracking. It can be implemented using recursion or an explicit stack.

- BFS $\rightarrow$ Queue
- DFS $\rightarrow$ Stack/Recursion

18. Hashing and Hash Table

Hashing converts a key into a value or index using a hash function. A hash table stores key-value information and is designed for fast average-case lookup.

Collision:

A collision occurs when two different keys map to the same hash-table position.

Common collision-handling techniques:

- Separate Chaining – maintain a collection such as a linked list/bucket at each position.
- Open Addressing – find another available position within the table.

19. Searching Algorithms

Linear Search

Checks elements one by one until the target is found or the data ends. Time complexity is O(n) in the worst case.

Binary Search

Works on sorted data. It repeatedly compares the target with the middle element and discards half of the search space.

Time complexity: O(log n).

20. Sorting Algorithms

Algorithm	Typical/General Idea	Common Complexity
Bubble Sort	Repeatedly swaps adjacent elements that are out of order	$O(n^2)$
Selection Sort	Repeatedly selects the minimum/maximum and places it	$O(n^2)$
Insertion Sort	Builds a sorted portion one element at a time	$O(n^2)$ worst case
Merge Sort	Divide, sort halves, then merge	O(n log n)
Quick Sort	Partition around a pivot and recursively sort	O(n log n) average; $O(n^2)$ worst case
Heap Sort	Uses a heap to repeatedly select an extreme element	O(n log n)

21. Recursion

Recursion is a technique in which a function calls itself to solve a smaller instance of the same problem. A recursive solution must have a base case to stop further calls.

```
int factorial(int n) {
    if (n == 0) return 1;
    return n * factorial(n - 1);
}
```

- Base case prevents infinite recursion.
- Recursive case reduces the problem.
- Function calls are managed using the call stack.

22. Dynamic Programming

Dynamic Programming (DP) solves problems by breaking them into overlapping subproblems and storing previously computed results so that they do not need to be recomputed.

- Memoization – top-down approach using stored results.
- Tabulation – bottom-up approach using a table/array.
- DP is useful when a problem has overlapping subproblems and optimal substructure.

23. Greedy Algorithm

A greedy algorithm makes the best-looking local choice at each step with the goal of obtaining a globally valid or optimal solution. Greedy methods are correct only for problems where the required greedy-choice property holds.

Examples include activity selection and some minimum spanning tree algorithms.

24. Common Data Structure Applications

Data Structure	Examples of Applications
Array	Tables, indexed collections, fixed-size data
Linked List	Dynamic sequences, playlists, memory structures
Stack	Undo, recursion, expression evaluation, DFS
Queue	Scheduling, buffering, BFS
Hash Table	Fast lookup, dictionaries, caches
Tree	Hierarchical data, searching, file systems
Heap	Priority queues, scheduling

Graph	Networks, maps, dependencies, social connections

25. Important Data Structures Interview Questions

1. What is a data structure and why is it needed?
2. What is the difference between an array and a linked list?
3. What is the difference between stack and queue?
4. Explain LIFO and FIFO with examples.
5. What are the applications of stacks?
6. What are the applications of queues?
7. What is a circular queue?
8. What is a binary tree?
9. What is a Binary Search Tree?
10. Why does inorder traversal of a BST produce sorted data?
11. What are preorder, inorder, and postorder traversals?
12. What is a heap? Explain min heap and max heap.
13. What is a graph?
14. What is the difference between BFS and DFS?
15. Why does BFS commonly use a queue?
16. Why can DFS be implemented using a stack?
17. What is hashing?
18. What is a collision in hashing?
19. What is the difference between adjacency matrix and adjacency list?
20. What is binary search and what condition is required for it?
21. Compare bubble sort, insertion sort, merge sort, and quick sort.
22. What is recursion?
23. What is dynamic programming?
24. What is memoization?
25. What is tabulation?
26. What is a greedy algorithm?

26. Quick Revision

- Array → Indexed linear collection
- Linked List → Nodes connected using references
- Stack → LIFO
- Queue → FIFO
- Deque → Insertion/deletion at both ends
- Tree → Hierarchical non-linear structure
- BST → Left values smaller, right values larger (under the standard unique-key rule)
- Heap → Complete binary tree with heap property

- Graph → Vertices + edges
- BFS → Queue
- DFS → Stack/Recursion
- Hash Table → Key-based fast average lookup
- Binary Search → Sorted data, O(log n)
- Merge Sort → Divide and merge, O(n log n)
- Quick Sort → Partition around pivot, average O(n log n)
- Recursion → Function calls itself with a base case
- Dynamic Programming → Store solutions to overlapping subproblems

27. Key Complexity Cheat Sheet

Operation/Algorithm	Typical Complexity
Array access	O(1)
Array linear search	O(n)
Linked-list traversal/search	O(n)
Stack push/pop	O(1)
Queue enqueue/dequeue	O(1) in standard implementations
Binary search	O(log n)
BST search	O(log n) average in a balanced tree; O(n) worst case
Heap insertion	O(log n)
Heap root access	O(1)
BFS	O(V + E) with adjacency lists
DFS	O(V + E) with adjacency lists
Merge sort	O(n log n)
Quick sort	O(n log n) average; $O(n^2)$ worst case